



Simulation de firmware IoT

Simulation de firmware IoT

1. Contexte

Dans les objets connectés (IoT), la **logique interne du système** est essentielle : elle définit le comportement du matériel (LED, capteur, alarme). Le processeur de l'objet ne peut se permettre d'être bloqué par une pause (`sleep`) pendant qu'une LED clignote, sinon il rate les messages du réseau ou les lectures de capteurs.

Avant de fabriquer un objet connecté, il est important de maîtriser cette **logique logicielle embarquée**.

Cet atelier vous permettra de **simuler un firmware IoT** en C, sans matériel, en gérant des **états système** et des **patterns de LED**.

*Nb: Dans un système informatique, un **firmware** (ou **micrologiciel**^[1], **microprogramme**^[1], ^[2], **microcode**, **logiciel interne** ou encore **logiciel embarqué**) est un programme intégré dans un appareil informatique (ordinateur, photocopieur, automate (API, APS), disque dur, routeur, appareil photo numérique, etc.) pour qu'il puisse fonctionner.*

fonctionner.

2. Objectifs pédagogiques

À l'issue de l'atelier, chaque participant sera capable de :

1. Définir et utiliser des **états système** (`enum`)
2. Implémenter une **boucle principale simulant le firmware**
3. Gérer des **patterns de clignotement différents** pour chaque état sans bloquer le programme
4. Programmer des **transitions automatiques d'état**
5. Structurer un code C modulaire et lisible.

3. Durée

- **1 heure :**
 - Introduction et explication : 10 min
 - Implémentation : 35 min
 - Tests et corrections : 10 min
 - Discussion / questions : 5 min

4. Sujet de l'atelier

Titre : Simulation logicielle d'un objet connecté à LED intelligente

Description :

Vous devez programmer le firmware simulé d'un boîtier connecté:

Cas d'un thermostat connecté.

Le système possède une seule **LED de statut** (simulée par du texte) qui doit réagir selon l'état du système.

Le système a 4 états :

État	Signification	Pattern LED simulé
STARTUP	Démarrage du système	3 clignotements rapides, puis passage à NORMAL
NORMAL	Fonctionnement normal	Clignotement lent régulier
WARNING	Surchauffe simulée	Clignotement rapide continu
ERROR	Panne critique	2 clignotements rapides + pause

Règles de fonctionnement :

1. **Le Temps** : Le cycle de simulation est de **100ms** (1 Tick/bip).
Tout le code se base sur ce "Tick/bip".
2. **Transitions** :
 - Le programme démarre en **STARTUP** .
 - Une fois l'animation **STARTUP** finie, il passe en **NORMAL** .
 - Ensuite, le système change d'état **toutes les 10 secondes** :

- **Affichage (optionnel)**: L'affichage doit se rafraîchir sur la même ligne (pas de défilement).

5. Méthodologie

Étape 1 : La structure de base

Mettez en place la structure de base avec la boucle infinie et la cadence de 100ms.

Astuce : Sous Windows utilisez `sleep(100)` , sous Linux/Mac `usleep(100000)` .

Étape 2 : La Machine à États

Définissez votre `enum` et implémentez le `switch/case` dans la boucle principale.

Pour l'instant, faites juste un `printf` simple pour vérifier que les états changent bien toutes les 10 secondes (100 Ticks).

Étape 3 : Les Patterns LED

C'est le cœur de l'algorithme. Remplacez les `printf` simples par la logique de la LED (ON/OFF).

- Utilisez l'opérateur modulo (`%`) sur le compteur de temps pour créer les rythmes.
- *Exemple pour clignoter toutes les secondes (10 ticks) :* `if (compteur % 10 < 5) { LED_ON; } else { LED_OFF; }`

Étape 4 : Affichage dynamique (Optionnel)

Utilisez le caractère `\r` au début de votre `printf` pour écraser la ligne précédente.

- *Exemple :* `printf("\r[ ETAT: %s ] LED: %s ", nom_etat, visuel_led);`

6. Ressources

Github: [Bien débiter sur GitHub : Ajouter son premier projet sans faux pas](#)

7. Livrables & Critères de réussite

1. Le code compile sans erreur.
2. On voit distinctement les rythmes de la LED changer selon l'état.
3. Le passage d'un état à l'autre se fait sans intervention humaine.
4. **Le code n'utilise pas de `sleep` à l'intérieur du `switch` (Interdiction formelle !).**

8. Pour aller plus loin (Bonus)

Si vous avez fini en avance :

- **Simuler un bouton** : Utilisez `kbhit()` (ou simulez une entrée aléatoire) pour forcer le passage en mode `ERROR` ou pour faire un "Reset" du système vers `STARTUP` .
- Intégrer cela à un véritable système avec une carte Arduino Uno
- **Logging** : Écrivez chaque changement d'état dans un fichier texte `system.log` avec le timestamp.

Code de base: